

pandas의 데이터 구조#2. DataFrame

학습내용

1. 데이터 프레임 생성
2. 데이터 프레임 구조
3. 데이터 프레임의 데이터 파악
4. 데이터 프레임 변경
5. 데이터 프레임 인덱싱
6. 데이터 프레임 인덱스 변경

```
In [1]: import numpy as np  
import pandas as pd
```

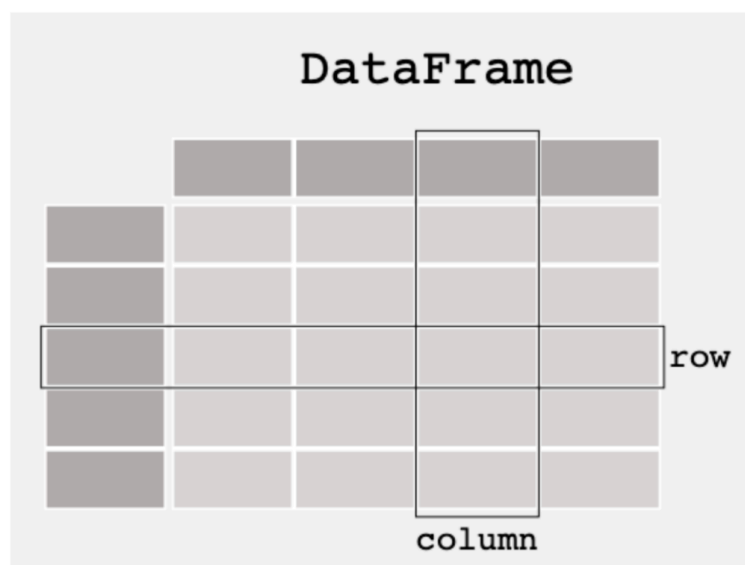
```
In [2]: print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [4]: from IPython.core.interactiveshell import InteractiveShell  
InteractiveShell.ast_node_interactivity = 'all' # 'last_expr'
```

DataFrame

- Pandas에서 주로 다루는 단위
- spreadsheet와 같은 개념
- Structured Data, Panel Data 또는 Tabular Data라고 부름
- pandas를 공부한다는 것 == dataframe의 사용법과 활용법을 배운다
- pandas를 잘 활용하면 대부분의 structured data를 자유자재로 다룰 수 있게 됨



데이터 프레임 특징

- 행과 열로 만들어지는 2차원 배열 구조
- 2차원 행렬 데이터에 인덱스를 붙인 것
- R의 데이터프레임에서 유래

- 데이터프레임의 각 열은 시리즈로 구성되어 있음

1. 데이터 프레임 생성

DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)함수 사용

- data : array-like, Iterable, dict, or scalar value
- index : index or array-like
- columns : index or array-like
- dtype : dtype

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

1) 리스트로 데이터 프레임 만들기

- DataFrame([[list1],[list2]])
- 각 list는 한 행으로 구성됨
- 행의 원소 개수가 다르면 None 값으로 저장

```
In [5]: pd.DataFrame([[1,2,3],[4,5,6],[7,8,9]])
```

```
Out[5]:
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

```
In [6]: pd.DataFrame([[1,2,3],[4,5,6],[7,8,9,10] ])
```

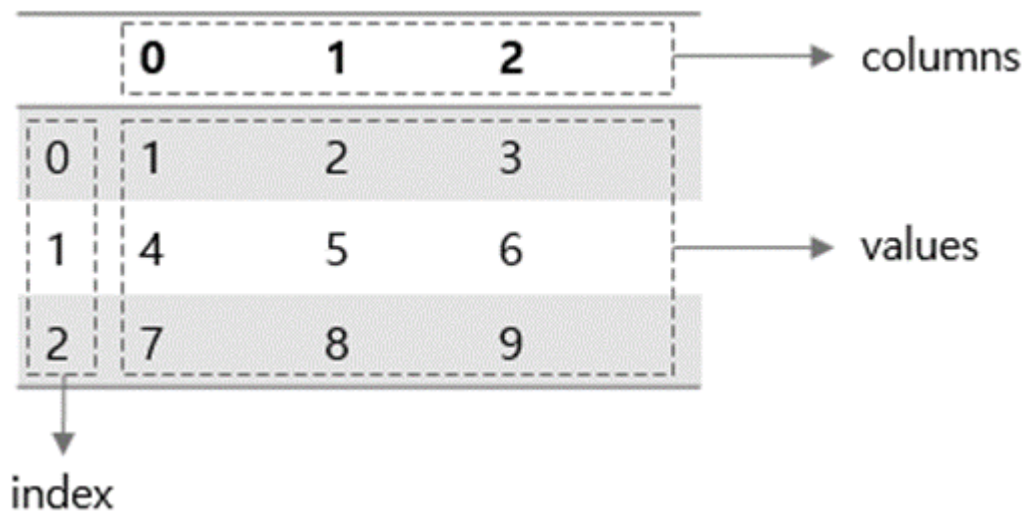
```
Out[6]:
```

	0	1	2	3
0	1	2	3	NaN
1	4	5	6	NaN
2	7	8	9	10.0

```
In [7]: df = pd.DataFrame([[1,2,3],[4,5,6],[7,8,9,10] ])
df.shape
```

```
Out[7]: (3, 4)
```

자동으로 생성된 index와 column을 갖는 DataFrame 데이터



2) 딕셔너리로 데이터프레임 생성

dict의 key → column

```
In [8]: data = {
    '2015': [9904312, 3448737, 2890451, 2466052],
    '2016': [9631482, 3393191, 2632035, 2000002],
    '2017': [9762546, 3512547, 2517680, 2456016],
    '2018': [9912345, 1513057, 2648043, 2504991]
}
```

data

```
Out[8]: {'2015': [9904312, 3448737, 2890451, 2466052],
        '2016': [9631482, 3393191, 2632035, 2000002],
        '2017': [9762546, 3512547, 2517680, 2456016],
        '2018': [9912345, 1513057, 2648043, 2504991]}
```

```
In [10]: df = pd.DataFrame(data=data)
df
```

```
Out[10]:
```

	2015	2016	2017	2018
0	9904312	9631482	9762546	9912345
1	3448737	3393191	3512547	1513057
2	2890451	2632035	2517680	2648043
3	2466052	2000002	2456016	2504991

```
In [11]: df.shape
df.index
df.columns
```

```
Out[11]: (4, 4)
```

```
Out[11]: RangeIndex(start=0, stop=4, step=1)
```

```
Out[11]: Index(['2015', '2016', '2017', '2018'], dtype='str')
```

3) 시리즈로 데이터 프레임 생성

각 Series의 인덱스 → column

```
In [12]: s1 = pd.Series(data=[100,101,102])
s2 = pd.Series(data=[200,201,202])
s3 = pd.Series(data=[300,301,302])
pd.DataFrame(data=[s1,s2,s3])
```

```
Out[12]:
```

	0	1	2
0	100	101	102
1	200	201	202
2	300	301	302

```
In [13]: s1 = pd.Series(data=[100,101,102], index='a b d'.split())
s2 = pd.Series(data=[200,201,202], index='a b k'.split())
s3 = pd.Series(data=[300,301,302], index='a b c'.split())
pd.DataFrame(data=[s1,s2,s3])
```

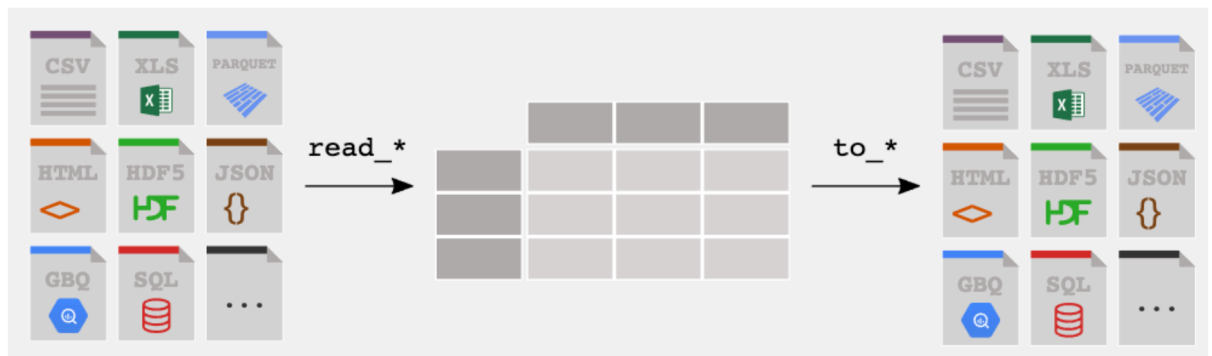
```
Out[13]:
```

	a	b	d	k	c
0	100.0	101.0	102.0	NaN	NaN
1	200.0	201.0	NaN	202.0	NaN
2	300.0	301.0	NaN	NaN	302.0

=> 시리즈의 인덱스는 시리즈로 구성된 데이터프레임의 컬럼인덱스가 됨

4) csv 데이터로 부터 Dataframe 생성

- 데이터 분석을 위해 dataframe을 생성하는 가장 일반적인 방법
- 데이터 소스로부터 추출된 csv(comma separated values) 파일로부터 생성
- **pandas.read_csv()** 함수 사용



read_csv() 함수 파라미터

- sep : 각 데이터 값을 구별하기 위한 구분자(separator) 설정
- header : header를 무시할 경우, None 설정
- index_col : index로 사용할 column 설정
- usecols : 실제로 dataframe에 로딩할 columns만 설정

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

타이타닉 데이터 다운로드

data 출처: <https://www.kaggle.com/datasets/hesh97/titanicdataset-traincsv/data>

```
In [14]: df_tit = pd.read_csv('data/train.csv')
df_tit
```

Out[14]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN
...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN

891 rows × 12 columns



```
In [15]: df_tit.head()
```

Out[15]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	

In [16]:

df_tit.tail()

Out[16]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	

In [17]:

df_tit.columns

Out[17]:

Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'], dtype='str')

In [18]:

df_tit.index

Out[18]: RangeIndex(start=0, stop=891, step=1)

```
In [20]: df_tit.size  
12*891
```

Out[20]: 10692

Out[20]: 10692

```
In [21]: df_tit.shape
```

Out[21]: (891, 12)

```
In [22]: df_tit.count()
```

```
Out[22]: PassengerId    891  
Survived              891  
Pclass               891  
Name                 891  
Sex                  891  
Age                  714  
SibSp                891  
Parch               891  
Ticket              891  
Fare                 891  
Cabin               204  
Embarked            889  
dtype: int64
```

```
In [23]: type(df_tit)
```

Out[23]: pandas.DataFrame

csv 파일의 일부 컬럼만 가지도록 데이터프레임을 다시 생성

```
In [24]: df_tit2 = pd.read_csv('data/train.csv', index_col='PassengerId')  
df_tit2.head()
```

Out[24]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emb
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	

In [25]: df_tit2.index

Out[25]: RangeIndex(start=1, stop=892, step=1, name='PassengerId')

In [26]: df_tit3 = pd.read_csv('data/train.csv', index_col='PassengerId',
usecols=['PassengerId', 'Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare'],
df_tit3.tail()

Out[26]:

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare
PassengerId							
887	0	2	male	27.0	0	0	13.00
888	1	1	female	19.0	0	0	30.00
889	0	3	female	NaN	1	2	23.45
890	1	1	male	26.0	0	0	30.00
891	0	3	male	32.0	0	0	7.75

데이터프레임을 csv파일로 저장 :

- <https://pandas.pydata.org> 의 API reference에서 함수 찾아보기
- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

In [27]: df4 = pd.DataFrame({'name': ['Raphael', 'Donatello'],
'mask': ['red', 'purple'],
'weapon': ['sai', 'bo staff']})

df4

Out[27]:

	name	mask	weapon
0	Raphael	red	sai
1	Donatello	purple	bo staff

In [28]: df4.index
df4.columns

Out[28]: RangeIndex(start=0, stop=2, step=1)

Out[28]: Index(['name', 'mask', 'weapon'], dtype='str')

In [29]: df4.index.name = 'id'
df4

Out[29]:

	name	mask	weapon
id			
0	Raphael	red	sai
1	Donatello	purple	bo staff

In [30]: df4.to_csv('data/sample.csv')

In [31]: df4.values

Out[31]: array([['Raphael', 'red', 'sai'],
 ['Donatello', 'purple', 'bo staff']], dtype=object)

In [33]: list(df4.items())

Out[33]: [(('name',
 id
 0 Raphael
 1 Donatello
 Name: name, dtype: str),
 ('mask',
 id
 0 red
 1 purple
 Name: mask, dtype: str),
 ('weapon',
 id
 0 sai
 1 bo staff
 Name: weapon, dtype: str))]

2. 데이터프레임의 구조

1) 행/열 방향 인덱스

인덱스(index)

- index 속성
- 각 아이템을 특정할 수 있는 고유의 값을 저장
- 복잡한 데이터의 경우, 멀티 인덱스로 표현 가능

컬럼(columns)

- columns 속성
- 각각의 특성(feature)을 나타냄
- 복잡한 데이터의 경우, 멀티 컬럼으로 표현 가능

DataFrame

Columns →

	Country	Capital	Population
1	Belgium	Brussels	11190846
2	India	New Delhi	1303171035
3	Brazil	Brasília	207847528

Index →

A two-dimensional labeled data structure with columns of potentially different types

```
>>> data = {'Country': ['Belgium', 'India', 'Brazil'],
            'Capital': ['Brussels', 'New Delhi', 'Brasília'],
            'Population': [11190846, 1303171035, 207847528]}

>>> df = pd.DataFrame(data,
                       columns=['Country', 'Capital', 'Population'])
```

- 출처 : <https://www.kdnuggets.com/2017/01/pandas-cheat-sheet.html>

2) 행/열 인덱스 출력

In [34]: `df_tit.head()`

Out[34]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

In [35]: `df_tit.index`

Out[35]: `RangeIndex(start=0, stop=891, step=1)`

```
In [36]: df_tit.columns
```

```
Out[36]: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',  
              'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],  
              dtype='str')
```

데이터프레임의 열방향 인덱스와 행방향 인덱스

The diagram illustrates the structure of a DataFrame. It shows a table with 3 rows and 3 columns. The first column is labeled 'index' and contains values 0, 1, and 2. The other two columns are labeled 'columns' and contain values 0, 1, and 2. The values in the other two columns are labeled 'values' and contain values 1, 2, 3, 4, 5, 6, 7, 8, and 9. The table is divided into three sections: the first section contains the index values, the second section contains the column headers, and the third section contains the data values.

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

3) 행/열 인덱스 이름 설정

- index.name
- columns.name

```
In [38]: print(df_tit.index.name)  
df_tit.index.name = 'attr'  
df_tit
```

None

Out[38]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
attr											
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN
...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN

891 rows × 12 columns



```
In [39]: print(df_tit.columns.name)
```

None

```
In [40]: df_tit.columns.name = 'cols'
df_tit.head()
```

Out[40]:

	cols	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
attr												
0		1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN
1		2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85
2		3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN
3		4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123
4		5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN

4) 데이터프레임의 값 접근

values 속성

In [43]:

```
df4
df4.values
df4.shape
```

Out[43]:

	name	mask	weapon
id			
0	Raphael	red	sai
1	Donatello	purple	bo staff

Out[43]:

```
array([[ 'Raphael', 'red', 'sai'],
       ['Donatello', 'purple', 'bo staff']], dtype=object)
```

Out[43]:

```
(2, 3)
```

3. 데이터프레임의 데이터 파악하기

- shape 속성 (row, column)
- describe() 함수 : 숫자형 데이터의 통계치 계산
- info() 함수 : 데이터 타입, 각 아이템의 개수 등 출력

shape : 데이터프레임의 행, 열 개수 출력

```
In [44]: df_tit.shape
```

```
Out[44]: (891, 12)
```

info() : 데이터프레임 개요

- 컬럼별 요약 : 데이터 개수(결측치제외한 데이터), 데이터유형(dtype)

```
In [45]: df_tit.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   PassengerId     891 non-null   int64  
1   Survived        891 non-null   int64  
2   Pclass          891 non-null   int64  
3   Name            891 non-null   str     
4   Sex             891 non-null   str     
5   Age             714 non-null   float64 
6   SibSp           891 non-null   int64  
7   Parch           891 non-null   int64  
8   Ticket          891 non-null   str     
9   Fare            891 non-null   float64 
10  Cabin           204 non-null   str     
11  Embarked        889 non-null   str     
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

```
In [46]: df4.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   name    2 non-null     str    
1   mask    2 non-null     str    
2   weapon  2 non-null     str    
dtypes: str(3)
memory usage: 180.0 bytes
```

describe(): 수치형 데이터의 기술통계 출력

```
In [47]: df_tit.describe()
```

```
Out[47]:
```

	cols	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count		891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean		446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std		257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min		1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%		223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%		446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%		668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max		891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [48]: df_tit.describe(include='all')
```

Out[48]:

cols	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	891.000000	
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	NaN	347082
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	0.381594	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	0.806057	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	0.000000	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	0.000000	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN

```
In [49]: df_tit.describe(include='str')
```

Out[49]:

cols	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Braund, Mr. Owen Harris	male	347082	G6	S
freq	1	577	7	4	644

```
In [50]: df_tit.describe(exclude='str')
```

Out[50]:

cols	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [51]: df_tit.describe(include=['float64'])
```

Out[51]:

	cols	Age	Fare
	count	714.000000	891.000000
	mean	29.699118	32.204208
	std	14.526497	49.693429
	min	0.420000	0.000000
	25%	20.125000	7.910400
	50%	28.000000	14.454200
	75%	38.000000	31.000000
	max	80.000000	512.329200

In [52]: df_tit.describe(include=['float64','str'])

Out[52]:

	cols	Name	Sex	Age	Ticket	Fare	Cabin	Embarked
	count	891	891	714.000000	891	891.000000	204	889
	unique	891	2	NaN	681	NaN	147	3
	top	Braund, Mr. Owen Harris	male	NaN	347082	NaN	G6	S
	freq	1	577	NaN	7	NaN	4	644
	mean	NaN	NaN	29.699118	NaN	32.204208	NaN	NaN
	std	NaN	NaN	14.526497	NaN	49.693429	NaN	NaN
	min	NaN	NaN	0.420000	NaN	0.000000	NaN	NaN
	25%	NaN	NaN	20.125000	NaN	7.910400	NaN	NaN
	50%	NaN	NaN	28.000000	NaN	14.454200	NaN	NaN
	75%	NaN	NaN	38.000000	NaN	31.000000	NaN	NaN
	max	NaN	NaN	80.000000	NaN	512.329200	NaN	NaN

In [53]: df_tit.describe(percentiles=[0.2,0.4,0.6,0.8], include=['float64'])

Out[53]:

	cols	Age	Fare
	count	714.000000	891.000000
	mean	29.699118	32.204208
	std	14.526497	49.693429
	min	0.420000	0.000000
	20%	19.000000	7.854200
	40%	25.000000	10.500000
	60%	31.800000	21.679200
	80%	41.000000	39.687500
	max	80.000000	512.329200

타이타닉 데이터 컬럼들

- PassengerId: 각 승객에게 주어진 고유 ID 번호
- Survived: 승객이 생존(1)했는지 사망(0)했는지 여부
- Pclass: 선실 등급(1:1등급, 2:2등급, 3:3등급)
- Name: 이름
- Sex: 승객의 성별
- Age: 승객의 나이
- SibSp: 형제자매/배우자의 수
- Parch: 부모/자녀의 수
- Ticket: 티켓 번호
- Fare: 티켓에 대해 지불한 금액
- Cabin: 객실 카테고리
- Embarked: 승객이 탑승한 항구(C = Cherbourg, Q = Queenstown, S = Southampton)

참고. 데이터 유형

- 수치형 데이터 : 평균, 분산, 최소값, 최대값, 중위수, 사분위수
 - 예. 타이타닉 데이터의 경우 age, SibSp
- 범주형 데이터 : 빈도, 비율
 - 예. 타이타닉 데이터의 경우 : sex, pclass
- PassengerID : 범주형 (명목형)
- Pclass : 범주형 (순서형)
- Sex : 범주형 (명목형)
- Age : 수치형 (비율형) => 사칙연산

4. 데이터프레임 변경

1) 데이터프레임 전치(transpose)

- pandas의 데이터 프레임은 전치를 포함해서 Numpy 2차원 배열의 대부분 속성이나 메서드를 지원.
- 전치 : 행과 열을 바꾸는 기능

```
In [54]: data = {
    '2015': [9904312, 3448737, 2890451, 2466052],
    '2016': [9631482, 3393191, 2632035, 2000002],
    '2017': [9762546, 3512547, 2517680, 2456016],
    '2018': [9912345, 1513057, 2648043, 2504991]
  }
df = pd.DataFrame(data=data)
df
```

```
Out[54]:
```

	2015	2016	2017	2018
0	9904312	9631482	9762546	9912345
1	3448737	3393191	3512547	1513057
2	2890451	2632035	2517680	2648043
3	2466052	2000002	2456016	2504991

```
In [55]: df.T
df.transpose()
```

```
Out[55]:
```

	0	1	2	3
2015	9904312	3448737	2890451	2466052
2016	9631482	3393191	2632035	2000002
2017	9762546	3512547	2517680	2456016
2018	9912345	1513057	2648043	2504991

```
Out[55]:
```

	0	1	2	3
2015	9904312	3448737	2890451	2466052
2016	9631482	3393191	2632035	2000002
2017	9762546	3512547	2517680	2456016
2018	9912345	1513057	2648043	2504991

2) 열(column) 추가

해당열이 있으면 내용 갱신, 열이 없으면 추가

- 열추가 : `df[열이름(key)]=values`
- 열 내용 갱신 : `df[열이름(key)]=values`

```
In [57]: df.index
df.columns
```

```
Out[57]: RangeIndex(start=0, stop=4, step=1)
```

```
Out[57]: Index(['2015', '2016', '2017', '2018'], dtype='str')
```

```
In [58]: df['2015'] # 컬럼을 기준으로 인덱싱 => 시리즈반환
```

```
Out[58]: 0    9904312
1    3448737
2    2890451
3    2466052
Name: 2015, dtype: int64
```

```
In [61]: df['평균'] = (df['2015'] + df['2016'] + df['2017'] + df['2018'])/4
df
```

```
Out[61]:
```

	2015	2016	2017	2018	평균
0	9904312	9631482	9762546	9912345	9802671.25
1	3448737	3393191	3512547	1513057	2966883.00
2	2890451	2632035	2517680	2648043	2672052.25
3	2466052	2000002	2456016	2504991	2356765.25

3) 열(column) 삭제

```
del df[삭제열]
```

```
In [62]: del df['평균']  
df
```

```
Out[62]:
```

	2015	2016	2017	2018
0	9904312	9631482	9762546	9912345
1	3448737	3393191	3512547	1513057
2	2890451	2632035	2517680	2648043
3	2466052	2000002	2456016	2504991

5. 데이터 프레임 인덱싱

- 열단위 인덱싱 []
- 인덱서(loc, iloc)를 사용하지 않는 행단위 인덱싱 [:]
- 개별 요소 접근 [열][행]

1) 열단위 인덱싱

- 열 라벨(컬럼명)을 키값으로 생각하고 인덱싱
- [] 기호 또는 df.열이름 을 이용
 - []는 열 위주 인덱싱이 원칙
- 인덱스로 라벨값을 하나 넣으면 시리즈 객체가 반환
- 라벨의 배열이나 리스트를 넣으면 부분적 df 가 반환

```
In [63]: data = {  
    '2015': [9904312, 3448737, 2890451, 2466052],  
    '2016': [9631482, 3393191, 2632035, 2000002],  
    '2017': [9762546, 3512547, 2517680, 2456016],  
    '2018': [9912345, 1513057, 2648043, 2504991]  
}  
df = pd.DataFrame(data, index=['서울', '부산', '인천', '대구'])  
df.columns.name = '연도'  
df.index.name = '도시'  
df
```

```
Out[63]:
```

	연도	2015	2016	2017	2018
도시					
서울	9904312	9631482	9762546	9912345	
부산	3448737	3393191	3512547	1513057	
인천	2890451	2632035	2517680	2648043	
대구	2466052	2000002	2456016	2504991	

```
In [64]: df['지역'] = ['수도권', '경상권', '경상권', '수도권']  
df
```

Out[64]:

	연도	2015	2016	2017	2018	지역
	도시					
	서울	9904312	9631482	9762546	9912345	수도권
	부산	3448737	3393191	3512547	1513057	경상권
	인천	2890451	2632035	2517680	2648043	경상권
	대구	2466052	2000002	2456016	2504991	수도권

```
In [65]: df['2015']
type(df['2015'])
```

Out[65]:

```
도시
서울      9904312
부산      3448737
인천      2890451
대구      2466052
Name: 2015, dtype: int64
```

Out[65]: pandas.Series

```
In [67]: # df.2016 # 문자열 2016이지만 정수로 인식하여 문법 에러 발생
df['2016']
```

Out[67]:

```
도시
서울      9631482
부산      3393191
인천      2632035
대구      2000002
Name: 2016, dtype: int64
```

열이름(컬럼명)이 문자열인 경우 위치인덱스 사용 불가

- 수치 인덱스를 사용할 수 없음
- keyerror 발생

```
In [68]: np.arange(1,10)
np.arange(1,10).reshape(3,3)
df_int = pd.DataFrame(np.arange(1,10).reshape(3,3))
df_int
```

Out[68]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])

Out[68]:

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Out[68]:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

```
In [69]: df_int.index
df_int.columns
```

Out[69]: RangeIndex(start=0, stop=3, step=1)

Out[69]: RangeIndex(start=0, stop=3, step=1)

```
In [70]: df_int[1]
```

```
Out[70]: 0    2
         1    5
         2    8
         Name: 1, dtype: int64
```

```
In [71]: 'A B C'.split()
df_int.columns = 'A B C'.split()
df_int
```

```
Out[71]: ['A', 'B', 'C']
```

```
Out[71]:
```

	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9

```
In [74]: # df_int[0]   # 컬럼이름이 문자열로 변경된 후에는 정수값인덱스 사용하면 KeyError 발생
```

```
In [76]: df_int['A']
df_int.A
```

```
Out[76]: 0    1
         1    4
         2    7
         Name: A, dtype: int64
```

```
Out[76]: 0    1
         1    4
         2    7
         Name: A, dtype: int64
```

추출한 열을 데이터프레임으로 받기

- 리스트로 인덱싱

```
In [77]: df
```

```
Out[77]:
```

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권
대구	2466052	2000002	2456016	2504991	수도권

```
In [80]: df['2016']
df[['2016']] # 열인덱싱에서 리스트로 컬럼을 지정하면 데이터프레임으로 반환
```

```
Out[80]:
```

도시	2016
서울	9631482
부산	3393191
인천	2632035
대구	2000002

```
Name: 2016, dtype: int64
```

```
Out[80]:
```

연도	2016
도시	
서울	9631482
부산	3393191
인천	2632035
대구	2000002

리스트를 사용하여 여러 개 열 추출

```
In [81]: df[['2016','2018']] # 열인덱싱에서 리스트로 컬럼들을 지정하면 데이터프레임으로 반환
```

```
Out[81]:
```

연도	2016	2018
도시		
서울	9631482	9912345
부산	3393191	1513057
인천	2632035	2648043
대구	2000002	2504991

열이름이 수치인 경우(위치인덱스)

```
In [84]: df_int.columns = range(3)
df_int
```

```
Out[84]:
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

```
In [85]: df_int[[0,2]]
```

```
Out[85]:
```

	0	2
0	1	3
1	4	6
2	7	9

2) 행 단위 인덱싱

- 슬라이싱(slicing) 사용
 - 열단위 인덱싱과 구분하기 위해
 - 인덱스 값이 문자(label)인 경우 문자 슬라이싱
- 인덱서(loc, iloc) 사용

```
In [86]: df
```

Out[86]:

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권
대구	2466052	2000002	2456016	2504991	수도권

한개의 행 추출

In [88]: df['서울':'서울']

Out[88]:

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권

In [89]: df['부산':'부산']

Out[89]:

연도	2015	2016	2017	2018	지역
도시					
부산	3448737	3393191	3512547	1513057	경상권

여러 행 추출

In [90]: df['부산':'대구']

Out[90]:

연도	2015	2016	2017	2018	지역
도시					
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권
대구	2466052	2000002	2456016	2504991	수도권

In [92]: df['부산':]
df['인천']

Out[92]:

연도	2015	2016	2017	2018	지역
도시					
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권
대구	2466052	2000002	2456016	2504991	수도권

Out[92]:

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권

In [93]:

```
df_int
df_int[0:0]
```

Out[93]:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Out[93]:

	0	1	2
--	---	---	---

In [94]:

```
df_int[0:1]
```

Out[94]:

	0	1	2
0	1	2	3

In [95]:

```
df_int[1:3]
```

Out[95]:

	0	1	2
1	4	5	6
2	7	8	9

In [97]:

```
df_int[:,2]
```

Out[97]:

	0	1	2
0	1	2	3
2	7	8	9

In [98]:

```
df_int[:,]
```

Out[98]:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

In [101...]

```
# df_int[[2:3, 1:2]] # 오류, 떨어져 있는 행을 인덱싱하는데 한계가 있음
```

3) 개별요소 접근 [열][행]

In [103... df

Out[103...

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권
부산	3448737	3393191	3512547	1513057	경상권
인천	2890451	2632035	2517680	2648043	경상권
대구	2466052	2000002	2456016	2504991	수도권

In [104... df['2015']['인천'] # numpy array인 경우 [행][열]

Out[104... np.int64(2890451)

In [106... df['서울':'서울']
df['서울':'서울']['2015']

Out[106...

연도	2015	2016	2017	2018	지역
도시					
서울	9904312	9631482	9762546	9912345	수도권

Out[106... 도시
서울 9904312
Name: 2015, dtype: int64

In [107... # 서울의 2015, 2017년 데이터 추출
df['서울':'서울'][['2015', '2017']]
df[['2015', '2017']]['서울':'서울']

Out[107...

연도	2015	2017
도시		
서울	9904312	9762546

Out[107...

연도	2015	2017
도시		
서울	9904312	9762546

In [110... df_int[0][1]
df_int[[0,2]][1:3]

Out[110... np.int64(4)

Out[110...

0	2
1	4 6
2	7 9

In [113... # df_int[0,1] # 사용불가

인덱싱 정리

- 데이터프레임은 열기준으로 접근 원칙 : df[열이름]
 - 행기준 접근을 위해서는 슬라이싱을 적용 : df[행이름:행이름]
 - 행과열을 같이 적용(개별요소) : df[열이름][행이름]
-
- 데이터프레임의 기본 행인덱싱은 슬라이싱방식을 사용하므로 연속된 위치만 접근 가능한 한계점이 있음
 - 이 단점을 보완한 인덱서 : loc[], iloc[]

```
In [115... df.loc['서울']
df.loc[['서울','대구']]
```

```
Out[115... 연도
2015      9904312
2016      9631482
2017      9762546
2018      9912345
지역      수도권
Name: 서울, dtype: object
```

```
Out[115... 연도      2015      2016      2017      2018      지역
도시
서울  9904312  9631482  9762546  9912345  수도권
대구  2466052  2000002  2456016  2504991  수도권
```

```
In [116... df
df.iloc[0,2]
```

```
Out[116... 연도      2015      2016      2017      2018      지역
도시
서울  9904312  9631482  9762546  9912345  수도권
부산  3448737  3393191  3512547  1513057  경상권
인천  2890451  2632035  2517680  2648043  경상권
대구  2466052  2000002  2456016  2504991  수도권
```

```
Out[116... np.int64(9762546)
```

6. 인덱스 변경

1) 행인덱스 변경

- 기본인덱스 : 행인덱스를 명시적으로 지정하지 않는 경우 0부터 1씩 증가하는 정수 인덱스 지정됨
- **set_index(keys, drop=True, append=False, inplace=False, ...)**
 - 기존의 행인덱스를 제거하고 지정한 열로 인덱스 변경
- **reset_index(level=None, drop=False, inplace=False, ...)**
 - 기존의 행인덱스를 제거하고 기본인덱스로 변경

```
In [117... # 행인덱스를 명시적으로 지정하지 않은 데이터프레임 생성
df = pd.DataFrame({'a':[1,3,2,4,5],
                   'b':[20,30,40,20,10],
                   'c':[1,5,7,3,2]})

df
```

```
Out[117...      a  b  c
0    1  20  1
1    3  30  5
2    2  40  7
3    4  20  3
4    5  10  2
```

```
In [118... df.columns
```

```
Out[118... Index(['a', 'b', 'c'], dtype='str')
```

```
In [119... df.index
```

```
Out[119... RangeIndex(start=0, stop=5, step=1)
```

행인덱스 변경 : set_index(열이름)

```
In [120... df2 = df.set_index('a')
df2
```

```
Out[120...      b  c
a
1    20  1
3    30  5
2    40  7
4    20  3
5    10  2
```

```
In [121... df2.index
```

```
Out[121... Index([1, 3, 2, 4, 5], dtype='int64', name='a')
```

```
In [122... df2.index.name
```

```
Out[122... 'a'
```

```
In [125... df.set_index('a')
df
df.set_index('a', inplace=True)
df
```

Out[125...

	b	c
a		
1	20	1
3	30	5
2	40	7
4	20	3
5	10	2

Out[125...

	a	b	c
0	1	20	1
1	3	30	5
2	2	40	7
3	4	20	3
4	5	10	2

Out[125...

	b	c
a		
1	20	1
3	30	5
2	40	7
4	20	3
5	10	2

기본인덱스(정수인덱스)로 변경 : reset_index(drop=False)

In [127...

```
df.reset_index(inplace=True)
df
```

Out[127...

	a	b	c
0	1	20	1
1	3	30	5
2	2	40	7
3	4	20	3
4	5	10	2

In [128...

```
df.set_index('a', inplace=True)
df
```

Out[128...

	b	c
a		
1	20	1
3	30	5
2	40	7
4	20	3
5	10	2

```
In [131... df.reset_index(drop=True, inplace=True)
df
```

Out[131...

	b	c
0	20	1
1	30	5
2	40	7
3	20	3
4	10	2

```
In [132... df.index
```

Out[132... RangeIndex(start=0, stop=5, step=1)

```
In [134... df_tit = pd.read_csv('data/train.csv')
df_tit.head(3)
```

Out[134...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

```
In [135... df_tit.set_index('PassengerId', inplace=True)
df_tit.head(2)
```

Out[135...

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarke
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	

In [136...

df_tit.index

Out[136...

RangeIndex(start=1, stop=892, step=1, name='PassengerId')

In [137...

df_tit.set_index('Name', inplace=True)
df_tit.head(2)

Out[137...

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
Name										
Braund, Mr. Owen Harris	0	3	male	22.0	1	0	A/5 21171	7.2500	NaN	S
Cumings, Mrs. John Bradley (Florence Briggs Thayer)	1	1	female	38.0	1	0	PC 17599	71.2833	C85	C

In [139...

df_tit.reset_index(drop=True)
df_tit.reset_index()

Out[139...

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	3	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	1	1	female	38.0	1	0	PC 17599	71.2833	C85	C
2	1	3	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	1	1	female	35.0	1	0	113803	53.1000	C123	S
4	0	3	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	211536	13.0000	NaN	S
887	1	1	female	19.0	0	0	112053	30.0000	B42	S
888	0	3	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	1	1	male	26.0	0	0	111369	30.0000	C148	C
890	0	3	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 10 columns

Out[139...

	Name	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	Braund, Mr. Owen Harris	0	3	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	1	1	female	38.0	1	0	PC 17599	71.2833	C85	C
2	Heikkinen, Miss. Laina	1	3	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	1	1	female	35.0	1	0	113803	53.1000	C123	S
4	Allen, Mr. William Henry	0	3	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
886	Montvila, Rev. Juozas	0	2	male	27.0	0	0	211536	13.0000	NaN	S
887	Graham, Miss. Margaret Edith	1	1	female	19.0	0	0	112053	30.0000	B42	S
888	Johnston, Miss. Catherine Helen "Carrie"	0	3	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	Behr, Mr. Karl Howell	1	1	male	26.0	0	0	111369	30.0000	C148	C
890	Dooley, Mr. Patrick	0	3	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

2) 새로운 색인 지정 : reindex()

- 새로운 색인(index)에 적합하도록 객체를 새로 생성
- 시계열 같은 순차적인 데이터를 재색인할 경우

```
Series.reindex(index=None, *, axis=None, method=None, copy=None,
               level=None, fill_value=None, limit=None, tolerance=None)
```

```
DataFrame.reindex(labels=None, *, index=None, columns=None, axis=None,
```

```
method=None, copy=None, level=None,  
fill_value=nan, limit=None, tolerance=None)
```

- labels : 색인으로 사용할 새로운 순서. Index 인스턴스나 다른 순차적인 파이썬 자료구조를 사용할 수 있음. Index는 복사가 이루어지지 않고 그대로 사용됨
- index : array-like, 전달된 시퀀스를 새로운 행(index) 레이블로 지정함
- columns : 전달된 시퀀스를 새로운 열 레이블로 지정함
- axis : int or str, 색인으로 사용할 축(행 또는 열)을 지정. 기본값은 행(index).
reindex(index=new_labels) 또는 reindex(columns=new_labels)와 같이 사용
- method : 채움 메서드. {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}
- fill_value : scalar, default np.nan. 재색인 과정 중에 새롭게 나타나는 비어 있는 데이터를 채우기 위한 값. 빈 곳의 결과에 null을 채워 넣으려면 fill_value='missing'을 이용
- limit : 전/후 보간(backward fill 또는 forward fill)을 위한 최대 갭 크기(채워 넣을 원소의 수)
- tolerance : 전/후 보간 시에 사용할 최대 갭 크기(값의 차이)
- level : Multiindex의 단계(level)에 단순 색인을 맞춤. 그렇지 않으면 Multiindex의 하위집합에 맞춤
- copy : True인 경우 새로운 색인이 이전 색인과 동일하더라도 기본 데이터를 복사함. False인 경우 새로운 색인이 이전 색인과 동일할 경우 복사하지 않음

시리즈 재색인

```
In [140...] s = pd.Series([4.5, 7.2, -5.3, 3.6], index=list('dabc'))  
s
```

```
Out[140...] d    4.5  
a    7.2  
b   -5.3  
c    3.6  
dtype: float64
```

```
In [141...] s.reindex('a b c d e'.split())  
s.reindex(['a','b','c','d','e'])
```

```
Out[141...] a    7.2  
b   -5.3  
c    3.6  
d    4.5  
e    NaN  
dtype: float64
```

```
Out[141...] a    7.2  
b   -5.3  
c    3.6  
d    4.5  
e    NaN  
dtype: float64
```

- 순차적인 데이터 재색인할 때 값을 보간하거나 채워야하는 경우

```
In [142...] s2 = pd.Series(['blue','purple','yellow'], index=[0,2,4])  
s2
```

```
Out[142...] 0    blue  
2    purple  
4    yellow  
dtype: str
```

```
In [144...] np.arange(6)  
s2.reindex(np.arange(6), method='ffill')  
s2.reindex(np.arange(6), method='bfill')
```


Out[144...] array([0, 1, 2, 3, 4, 5])

Out[144...] 0 blue
1 blue
2 purple
3 purple
4 yellow
5 yellow
dtype: str

Out[144...] 0 blue
1 purple
2 purple
3 yellow
4 yellow
5 NaN
dtype: str

데이터프레임 재색인

```
In [145...] df = pd.DataFrame(np.arange(9).reshape(3,3),  
                        index='a c d'.split(),  
                        columns='A B C'.split())  
df
```

Out[145...]

	A	B	C
a	0	1	2
c	3	4	5
d	6	7	8

- 행 인덱스 재색인

```
In [147...] df.reindex(index='a b c d'.split())  
df.reindex(index='a b c d'.split(), method='ffill')
```

Out[147...]

	A	B	C
a	0.0	1.0	2.0
b	NaN	NaN	NaN
c	3.0	4.0	5.0
d	6.0	7.0	8.0

Out[147...]

	A	B	C
a	0	1	2
b	0	1	2
c	3	4	5
d	6	7	8

- 열 인덱스 재색인

```
In [149...] df.reindex(columns=['B','D','C'])  
df.reindex(columns=['B','D','C'], method='bfill')
```

Out[149...

	B	D	C
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Out[149...

	B	D	C
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

```
In [151... df.reindex(['B','D','C'], axis='columns')
df.reindex(['a','b','d','c'], axis='index')
```

Out[151...

	B	D	C
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Out[151...

	A	B	C
a	0.0	1.0	2.0
b	NaN	NaN	NaN
d	6.0	7.0	8.0
c	3.0	4.0	5.0

```
In [157... index = ['Firefox', 'Chrome', 'Safari', 'IE10', 'Konqueror']
df = pd.DataFrame({'http_status': [200, 200, 404, 404, 301],
                    'response_time': [0.04, 0.02, 0.07, 0.08, 1.0]},
                    index=index)
df
```

Out[157...

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

```
In [158... new_index = ['Safari', 'Iceweasel', 'Comodo Dragon', 'IE10', 'Chrome']
df.reindex(new_index)
df.info()
```

Out[158...

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02

<class 'pandas.DataFrame'>
Index: 5 entries, Firefox to Konqueror
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- -
0 http_status 5 non-null int64
1 response_time 5 non-null float64
dtypes: float64(1), int64(1)
memory usage: 292.0+ bytes

In [155...

```
df.reindex(new_index, fill_value=0)  
df.reindex(new_index, fill_value=0).info()
```

Out[155...

	http_status	response_time
Safari	404	0.07
Iceweasel	0	0.00
Comodo Dragon	0	0.00
IE10	404	0.08
Chrome	200	0.02

<class 'pandas.DataFrame'>
Index: 5 entries, Safari to Chrome
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- -
0 http_status 5 non-null int64
1 response_time 5 non-null float64
dtypes: float64(1), int64(1)
memory usage: 120.0+ bytes

In [159...

```
df.reindex(new_index, fill_value='missing')  
df.reindex(new_index, fill_value='missing').info()
```

Out[159...

	http_status	response_time
Safari	404	0.07
Iceweasel	missing	missing
Comodo Dragon	missing	missing
IE10	404	0.08
Chrome	200	0.02

```
<class 'pandas.DataFrame'>
Index: 5 entries, Safari to Chrome
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   http_status      5 non-null      object
1   response_time    5 non-null      object
dtypes: object(2)
memory usage: 120.0+ bytes
```

```
In [161... df.reindex(['http_status', 'user-agent'], axis='columns')
```

```
Out[161...
      http_status  user-agent
Firefox         200         NaN
Chrome          200         NaN
Safari          404         NaN
IE10            404         NaN
Konqueror       301         NaN
```

```
In [162... date_index = pd.date_range('1/1/2010', periods=6, freq='D')
df2 = pd.DataFrame({"prices": [100, 101, np.nan, 100, 89, 88]},
                    index=date_index)
df2
```

```
Out[162...
      prices
2010-01-01  100.0
2010-01-02  101.0
2010-01-03   NaN
2010-01-04  100.0
2010-01-05   89.0
2010-01-06   88.0
```

```
In [165... date_index2 = pd.date_range('12/29/2009', periods=10, freq='D')
date_index2
```

```
Out[165... DatetimeIndex(['2009-12-29', '2009-12-30', '2009-12-31', '2010-01-01',
                '2010-01-02', '2010-01-03', '2010-01-04', '2010-01-05',
                '2010-01-06', '2010-01-07'],
                dtype='datetime64[us]', freq='D')
```

```
In [167... df2.reindex(date_index2, method='bfill')
```

Out[167...

prices	
2009-12-29	100.0
2009-12-30	100.0
2009-12-31	100.0
2010-01-01	100.0
2010-01-02	101.0
2010-01-03	NaN
2010-01-04	100.0
2010-01-05	89.0
2010-01-06	88.0
2010-01-07	NaN

3) 인덱스 이름 변경

- 행인덱스(index) 또는 열인덱스(columns)의 값 변경
- `rename(index= { }, columns={ }, ..., inplace=False,...)` 함수
 - 딕셔너리 형식으로 지정
 - 행인덱스(index) 이름 변경 : `rename(index={행이름1:변경값1, 행이름1:변경값1, ...})`
 - 열인덱스(columns) 이름 변경 : `rename(columns={열이름1:변경값1, 열이름2:변경값2, ... })`

열인덱스 이름 변경

In [168...

```
df
```

Out[168...

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

In [169...

```
df.columns
```

Out[169...

```
Index(['http_status', 'response_time'], dtype='str')
```

In [171...

```
df.reset_index(inplace=True)  
df
```

Out[171...

	index	http_status	response_time
0	Firefox	200	0.04
1	Chrome	200	0.02
2	Safari	404	0.07
3	IE10	404	0.08
4	Konqueror	301	1.00

In [174...

```
df.rename(columns={'index':'Browser'}, inplace=True)
df
```

Out[174...

	Browser	http_status	response_time
0	Firefox	200	0.04
1	Chrome	200	0.02
2	Safari	404	0.07
3	IE10	404	0.08
4	Konqueror	301	1.00

행인덱스 이름 변경

In [177...

```
df.rename(index={0:'I',1:'J',2:'K',3:'L',4:'M'}, inplace=True)
df
```

Out[177...

	Browser	http_status	response_time
I	Firefox	200	0.04
J	Chrome	200	0.02
K	Safari	404	0.07
L	IE10	404	0.08
M	Konqueror	301	1.00

In [178...

```
df.index = range(5)
df
```

Out[178...

	Browser	http_status	response_time
0	Firefox	200	0.04
1	Chrome	200	0.02
2	Safari	404	0.07
3	IE10	404	0.08
4	Konqueror	301	1.00

In [179...

```
df.index
```

Out[179...

RangeIndex(start=0, stop=5, step=1)

In [182...

```
df.rename(index=str).index  
df.rename(index=str)
```

Out[182...

```
Index(['0', '1', '2', '3', '4'], dtype='str')
```

Out[182...

	Browser	http_status	response_time
0	Firefox	200	0.04
1	Chrome	200	0.02
2	Safari	404	0.07
3	IE10	404	0.08
4	Konqueror	301	1.00

학습내용

1. 데이터프레임 생성

- pd.DataFrame()
- 2차원 리스트, 딕셔너리, 시리즈, array_like
- csv 파일(pd.read_csv())
- 데이터프레임을 csv 저장: pd.to_csv()

2. 데이터프레임 구조

- 2차원 tabular 형식 (행, 열)
- columns, index, values

3. 데이터프레임의 데이터 파악

- info() : 데이터 변수, 결측치, 데이터 유형 등 개요
- describe() : 수치형데이터의 요약통계량(count, mean, std, min, 25%, 50%, 75%, max)
 - describe(include=all) : 범주형데이터와 수치형데이터 요약통계량
 - 범주형 데이터는 top, freq, unique
 - include, exclude : 데이터유형으로 지정
 - describe(include=['int64'])
- head(), tail()
- size, shape

4. 데이터프레임 변경

- 컬럼 삭제, 삽입

5. 데이터프레임 인덱싱

- 열 인덱싱 []
- 행 인덱싱 [:]
- 개별요소 인덱싱 [열][행:행]
- loc(), iloc() 인덱서

6. 데이터프레임 인덱스(index) 변경

- set_index()

- `reset_index()`

7. 재색인(reindex)

- `reindex()`

8. 데이터프레임의 행/열 이름 변경

- `rename(columns={old:new}, index={old:new})`
-